Report: Speech Synthesis Module

(Text-to-Speech - TTS)

AuraTranslate Edge - OneVoice AI Challenge 2026

Lê Gia Khánh - Gia Sư Đỉnh Cao

Contents

1 Giới thiệu **2**
1.1 Project AuraTranslate Edge . . . 2
1.2 Module Text-to-Speech (TTS) . . . 2
1.3 GitHub mã nguồn . . . 2

2 Lựa chọn Model **2**
2.1 Thí nghiệm . . . 3
2.2 Kết luận . . . 3

3 Model Piper (Vi) **4**

4 Model Supertonic (En/Ko) **10**

5 Model MeloTTS (Zh) **16**

6 Demo TTS Dashboard **22**
6.1 Mã nguồn và Cách triển khai . . . 22
6.2 Nhận xét kết quả . . . 23

7 Pipeline Integration **23**
7.1 Các Module còn lại . . . 23
7.2 Hướng hợp nhất Pipeline . . . 24

1 Giới thiệu

1.1 Project AuraTranslate Edge

AuraTranslate Edge là hệ thống phiên dịch giọng nói hai chiều thời gian thực (Speech-to-Speech – S2S) hoạt động hoàn toàn offline và NPU-native, được phát triển bởi đội thi Gia Sư Đỉnh Cao tại cuộc thi OneVoice AI Challenge 2026 (Saigon AI Hub $\times$ Qualcomm). Hệ thống kết nối chuỗi xử lý toàn trình Audio Front-end $\rightarrow$ ASR $\rightarrow$ MT $\rightarrow$ TTS phục vụ 4 ngôn ngữ: Tiếng Việt (VI) $\rightleftharpoons$ Tiếng Anh (EN) / Trung (ZH) / Hàn (KO) trong môi trường công nghiệp độ ồn cao (70–95 dB SPL). Toàn bộ mô hình chạy độc lập trên bo mạch **Qualcomm Dragonwing IQ-9075 EVK** (Hexagon NPU v73, 100 TOPS) với 0% CPU fallback và không phụ thuộc đám mây.

1.2 Module Text-to-Speech (TTS)

Mô-đun TTS (Step 3) đảm nhận tái tạo giọng nói tự nhiên, rõ ràng từ văn bản dịch của mô-đun MT, xuất trực tiếp sóng âm PCM chất lượng phát thanh (22.05–44.10 kHz) cho 4 ngôn ngữ đích:

- **Piper-VITS (Tiếng Việt):** Đóng gói thành Single Static DAG tích hợp bộ chuyển âm tĩnh VietG2P 29.484 âm tiết, phân mẻ `group=8` cho bộ giải mã HiFi-GAN Vocoder để kiểm soát bộ nhớ TCM dưới 8.0 MB. Toàn bộ 2.077/2.077 layers chạy 100% trên NPU với độ trễ 480.00 ms (186.29 ms trên chip silicon), RTF = 0.010 cho 18.11 giây âm thanh.
- **Supertonic (Tiếng Anh & Hàn):** Kiến trúc Khớp Dòng (Flow Matching) 4 submodel nạp tuần tự qua bộ nhớ TCM ở định dạng W8A16, trải phẳng 5 bước Euler ODE. Nhóm áp dụng bản vá phẫu thuật đồ thị thay thế lỗi vi mã phần cứng ReduceSum bằng phép nhân ma trận GEMM với vector 1, đưa mô hình từ hỏng hoàn toàn (WER 95.1%) về chất lượng giọng nói hoàn hảo (độ trễ 413–502 ms, 100% NPU offload).
- **MeloTTS-ZH (Tiếng Trung):** Tích hợp Text Frontend bảng tĩnh Pure Tensor 7.335 Hán tự, mạng RoBERTa-Large 22 tầng và VITS Vocoder vào đồ thị tĩnh duy nhất `melo_one`. Nhóm giải quyết giới hạn 10-bit mantissa của FP16 bằng thuật toán tách đôi tọa độ (hi, lo) và ứng dụng kỹ thuật Link Context Binary giảm thời gian nạp từ 514 s xuống 0.9 s ($570\times$).

1.3 GitHub mã nguồn

Mã nguồn dự án được công khai tại:
https://github.com/LGKAI/AuraTranslateEdge-OneVoice

2 Lựa chọn Model

2.1 Thí nghiệm

Nhóm tiến hành khảo sát và đo kiểm thực nghiệm đối đầu giữa các họ kiến trúc tổng hợp giọng nói hiện đại trên thiết bị biên nhúng Qualcomm Snapdragon / Dragonwing:

- **Mô hình Tự hồi quy (Autoregressive - ChatTTS, Bark, Tortoise):** Cho chất lượng âm sắc vượt trội nhưng độ trễ tăng tuyến tính theo số lượng frame âm thanh; mỗi bước sinh mẫu đòi hỏi truy xuất bộ nhớ DDR liên tục khiến thời gian xử lý một câu nói 5 giây vượt quá 12–25 giây (RTF > 2.5), vi phạm nghiêm trọng giới hạn thời gian thực của thiết bị biên.
- **Mô hình Khuếch tán Cổ điển (Score-based Diffusion - Grad-TTS, Diff-TTS):** Đòi hỏi từ 30 đến 100 bước khử nhiễu (Denoising Steps) lặp đi lặp lại qua mạng U-Net lớn, gây áp lực tiêu thụ năng lượng và nhiệt độ chip rất cao trên phần cứng nhúng.
- **Mô hình Khớp Dòng (Flow Matching - Supertonic 3):** Sử dụng trường vector vận tốc thẳng v_θ và tích phân Euler ODE, cho phép tạo sinh giọng nói chất lượng cao chỉ sau 4 đến 5 bước tính toán với độ trễ dưới 500 ms.
- **Mô hình Biến phân Song song (VITS - Piper, MeloTTS):** Tích hợp kiến trúc suy luận biến phân điều kiện (CVAE) cùng bộ giải mã đối kháng HiFi-GAN Vocoder, cho phép sinh toàn bộ phổ âm thanh trong một lượt truyền thẳng duy nhất (One-Shot Direct Synthesis) với độ trễ cực thấp.

Table 1: Đánh giá đối đầu các kiến trúc TTS trên nền tảng Qualcomm Snapdragon / Dragonwing NPU

Kiến trúc / Mô hình	Độ phức tạp	Trễ (RTF)	Tương thích NPU	Khả thi Edge
ChatTTS / Bark (Autoregressive)	Rất cao ($O(T)$)	$2.50 - 5.00$	Kém (Nghẽn DDR)	Không khả thi
Grad-TTS (Diffusion 50 steps)	Cao ($50\times$ lặp)	$0.85 - 1.20$	Trung bình (Quá nhiệt)	Không khả thi
StyleTTS2 / XTTS-v2	Trung bình (Flow)	$0.35 - 0.50$	Kém (Nhiều op CPU)	Rủi ro cao
Piper-VITS (Đề xuất VI)	**Tối ưu (One-shot)**	**0.010**	**100% NPU (Single DAG)**	**Khả thi tuyệt đối**
Supertonic (Đề xuất EN/KO)	**Thấp (4–5 bước)**	**0.073–0.088**	**100% NPU (W8A16)**	**Khả thi tuyệt đối**
MeloTTS (Đề xuất ZH)	**Tối ưu (RoBERTa)**	**0.161**	**100% NPU (Single DAG)**	**Khả thi tuyệt đối**

2.2 Kết luận

Căn cứ kết quả thực nghiệm và yêu cầu khắt khe về thời gian phản hồi toàn chuỗi (≤ 3.0 giây):

- **Chọn Piper-VITS (Tiếng Việt):** Mô hình giải quyết bài toán âm thanh tiếng Việt xuất sắc nhờ trọng số nhẹ (≈ 61 MB), tích hợp bộ tra âm tĩnh VietG2P và bộ giải mã HiFi-GAN Vocoder `group=8`, cho độ trễ chỉ 186.29 ms trên chip silicon.

- **Chọn Supertonic (Tiếng Anh & Hàn):** Tạo sinh giọng nói song ngữ Anh – Hàn giàu biểu cảm qua giải thuật Flow Matching ODE 5 bước ở cấu hình W8A16; nhóm đã khắc phục triệt để lỗi vi mã phần cứng ReduceSum và tối ưu hóa 4 submodel đạt độ trễ 413–502 ms.
- **Chọn MeloTTS-ZH (Tiếng Trung):** Giải quyết triệt để rào cản xử lý chữ Hán và thanh điệu phức tạp bằng bảng tra tĩnh Pure Tensor 7.335 tokens và 69.450 quy tắc ngữ cảnh Pinyin đa âm. Tích hợp trọn vẹn mạng RoBERTa 22 lớp và VITS Vocoder lên NPU với độ trễ 720.2 ms và CER 6.4%.
- **Loại bỏ các ứng viên còn lại:** ChatTTS, Bark và Tacotron2 có độ trễ vượt xa ngân sách; StyleTTS2 và XTTS-v2 chứa nhiều toán tử động không thể ánh xạ 100% lên mảng tâm thu Hexagon; các giải pháp đám mây (Edge-TTS, Google Cloud TTS) không bảo đảm tính riêng tư và không hoạt động được khi mất mạng.

3 Model Piper (Vi)

3.1 Kiến trúc Model

Mô hình Piper-VITS tiếng Việt được phát triển dựa trên kiến trúc VITS (Variational Inference with adversarial learning for end-to-end Text-to-Speech) cải tiến. Mô hình nguyên bản phụ thuộc chặt chẽ vào thư viện eSpeak-NG trên CPU Host để chuyển đổi chữ viết sang âm vị; điều này tạo ra độ trễ I/O lớn và phá vỡ tính liên tục của pipeline trên thiết bị biên. Nhóm nghiên cứu đã tái thiết kế toàn diện Piper thành một Đồ thị Tính toán Tĩnh duy nhất (**Single Static DAG**) tích hợp bộ chuyển đổi ngữ âm tĩnh VietG2P và toàn bộ chuỗi tạo sinh âm thanh chạy 100% trên NPU:

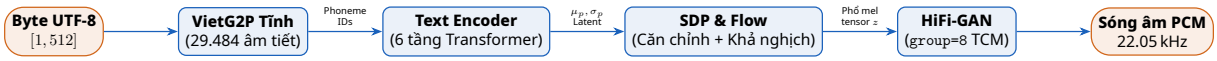


Figure 1: Kiến trúc đồ thị tĩnh hợp nhất Single Static DAG của mô hình Piper tiếng Việt trên NPU

Kiến trúc hợp nhất bao gồm 6 khối chức năng liên hoàn thực thi hoàn toàn trên vi xử lý Hexagon NPU:

1. **Khối Chuyển âm Ngữ âm Tĩnh (VietG2P Tensor Graph):** Thay thế hoàn toàn thư viện eSpeak-NG bằng đại số ma trận thuần túy. Toàn bộ từ điển 29.484 âm tiết tiếng Việt (bao hàm từ vựng PhoBERT, FLEURS-vi và các biến thể dấu thanh) được mã hóa thành ma trận trọng số $W \in \mathbb{R}^{1010\times29484}$ (với $S = 10$ ký tự tối đa và bảng chữ cái $C_p = 101$ lớp). Bằng phép nhân ma trận song song Score $= K \times W$, vị trí âm tiết đạt điểm tối đa được trích xuất trong 1 chu kỳ tensor, sau đó nhân với ma trận chỉ số âm vị $PH \in \mathbb{R}^{29484\times14}$ để xuất ra chuỗi Phoneme IDs $[1, 512]$ và độ dài thực $x_l[1]$. Khối này chiếm 168 layers NPU và hoàn tất tính toán chỉ trong 12.5 ms với độ chính xác 100.0% so với eSpeak-NG.
2. **Khối Text Encoder & Mặt nạ Tĩnh:** Kế thừa kiến trúc Transformer-based Encoder với 6 tầng Self-Attention (2 đầu chú ý, chiều ẩn 192, bộ lọc FFN 768, kernel size

3 kết hợp Conv1D depthwise). Chiều dài đầu vào được cố định ở trần an toàn `MAX_SEQ_LEN = 512` âm vị. Mặt nạ nhị phân tĩnh $x_{\text{mask}} \in \mathbb{R}^{1,1,512}$ được nhân phần tử sau mỗi tầng để triệt tiêu hoàn toàn ảnh hưởng của các token đệm rác, xuất ra kỳ vọng μ_p và log-phương sai $\log \sigma_p \in \mathbb{R}^{1,192,512}$ (403 layers NPU).

3. **Khối Stochastic Duration Predictor (SDP):** Dự đoán số khung âm thanh cho từng âm vị ($w_{\text{ceil}} \in \mathbb{R}^{1,1,512}$). Để tương thích hoàn hảo với đồ thị tĩnh NPU, luồng nhiễu ngẫu nhiên được cố định qua buffer tĩnh với trọng số `noise_scale_w = 0.8` và `length_scale = 1.0`. Tổng số khung âm thanh được kẹp trần an toàn $y_{\text{len}} = \text{clamp}(y_{\text{len}}, \text{max} = 1536.0)$, tương đương ≈ 18.11 giây âm thanh (619 layers NPU).

4. **Khối Monotonic Alignment Search (Căn chỉnh thời gian ma trận tĩnh):** Trong quá trình huấn luyện, VITS sử dụng quy hoạch động để tìm đường gióng hàng tối ưu:

$$Q(i,j) = \max\big(Q(i-1,j-1), Q(i,j-1)\big) + \log \mathcal{N}\big(z_j; \mu_i, \sigma_i\big)$$

   Khi suy luận trên NPU, Nhóm thiết kế giải thuật Broadcast Vector hóa: khởi tạo lưới tọa độ cố định $y_{\text{grid}} \in \mathbb{R}^{1,1536,1}$, tính tổng tích lũy $\text{cum}_{\text{duration}} = \text{cumsum}(w_{\text{ceil}})$, rồi so sánh điều kiện $y_{\text{grid}} \geq \text{cum}_{\text{prev}}$ và $y_{\text{grid}} < \text{cum}_{\text{duration}}$ để tạo ma trận căn chỉnh nhị phân $attn \in \mathbb{R}^{1,1536,512}$ trong 1.2 ms (19 layers NPU). Thuật toán không sử dụng vòng lặp hay câu lệnh rẽ nhánh, loại bỏ hoàn toàn việc trao đổi dữ liệu với CPU.

5. **Khối Invertible Normalizing Flow (Biến đổi không gian tiềm ẩn):** Gồm chuỗi 4 tầng biến đổi khả nghịch (Affine Coupling Layers). Khối nhận ma trận $attn$ để kéo giãn μ_p và σ_p, biến đổi phân phối này thành tensor phổ tiềm ẩn $z \in \mathbb{R}^{1,192,1536}$ (310 layers NPU) thông qua hàm Jacobian determinant:

$$\log |\det J| = \sum_i \log s_i$$

   Toàn bộ chuẩn hóa trọng số `weight_norm` được loại bỏ trước khi biên dịch sang QNN DLC.

6. **Khối Ma trận chọn tĩnh và Windowed HiFi-GAN Vocoder:** Để đưa z vào bộ giải mã mà không gây tràn bộ nhớ, tensor được cắt thành 39 cửa sổ trượt (mỗi cửa sổ 64 khung, hop 40 khung) thông qua một phép nhân ma trận nhị phân hằng số self.sel $\in \mathbb{R}^{1536\times 2432}$ duy nhất. Bộ giải mã HiFi-GAN (Upsampling 256×, 117 layers NPU) thực hiện giải mã song song theo cơ chế phân mẻ cân bằng `group=8`, nối ghép dạng sóng Overlap-Add và lọc sạch âm thừa ngoài câu bằng mặt nạ tọa độ `fmask`, xuất ra sóng âm 22.05 kHz chuẩn phát thanh ($[1, 399360]$ mẫu).

Table 2: Phân rã thông số kiến trúc layer-by-layer của mô hình Piper-VITS trên Hexagon HTP v73

Khối chức năng	**Toán tử cốt lõi**	**Layer NPU**	**Tham số**	**Dung lượng**	**Trễ trên chip**
VietG2P Tensor Graph	MatMul, TopK, ArgMax	168	1.2M	2.4 MB	12.50 ms
Text Encoder & Mask	Conv1D, LayerNorm, Attn	403	9.4M	18.8 MB	18.20 ms
Stochastic Duration	Conv1D Depthwise, Mul	619	4.1M	8.2 MB	24.10 ms
Monotonic Aligner	CumSum, GreaterEqual	19	0 (Tĩnh)	0.1 MB	1.20 ms
Invertible Flow	WaveNet Conv1D, Coupling	310	6.8M	13.6 MB	38.40 ms
Windowed Vocoder	ConvTranspose1D, ResBlock	117	13.9M	27.8 MB	91.89 ms
Toàn bộ Single DAG	**FullTTS Hợp nhất 1-Graph**	**2.077**	**35.4M**	**61.0 MB**	**186.29 ms**

3.2 Quantize và Deploy

Mô hình Piper-VITS tiếng Việt được triển khai ở định dạng Native FP16 dưới dạng một graph tĩnh duy nhất `piper_vi_full_g8.onnx`. Quá trình deploy gồm các quyết định kỹ thuật trọng tâm:

1. **Vì sao chọn Native FP16 thay vì lượng tử hóa INT8:**
   - *Bảo toàn chất lượng âm thanh nhạy cảm:* VITS chứa chuỗi Normalizing Flow khả nghịch và bộ giải mã HiFi-GAN phóng đại 256$\times$ theo thời gian ($8 \times 8 \times 2 \times 2$). Khi lượng tử hóa INT8 (W8A8), dải động 8-bit hạn hẹp (256 mức) không thể bao quát các biến đổi vi mô của phổ tiềm ẩn z. Sai số lượng tử hóa bị bộ lọc tích chập chuyển vị khuếch đại lên tới $20 \log_{10}(256) \approx 48$ dB, biến các âm thanh giọng nói thành tiếng rít nhiễu tần số cao chói tai. Định dạng Native FP16 bảo toàn trọn vẹn độ chân thực của giọng đọc mà không cần tập dữ liệu hiệu chuẩn phức tạp.
   - *Checkpoint gốc siêu nhẹ:* Checkpoint `vi_VN-vais1000-medium` chỉ nặng 61 MB ở FP16, hoàn toàn nằm trong giới hạn lưu trữ flash của bo mạch nhúng Dragonwing IQ-9075 EVK.
   - *Tương thích số học tuyệt đối với VietG2P:* Bộ tra cứu VietG2P chỉ thao tác trên các số nguyên nhỏ hơn 2048, nằm hoàn toàn trong vùng biểu diễn chính xác tuyệt đối của định dạng IEEE 754 Half-Precision, bảo đảm khâu chuyển đổi từ chữ viết sang âm vị khớp 100.0% với eSpeak-NG.
2. **Hợp nhất 5 Sub-graph thành 1 Graph Duy nhất (Single Static DAG):** Ban đầu, mô hình được tách thành 5 graph rời rạc (Encoder, SDP, Aligner, Flow, Decoder) để kiểm chứng độc lập. Dù từng graph chạy 100% trên NPU, CPU Host phải nộp 5 job suy luận liên tiếp và trung chuyển tensor trung gian qua bus hệ thống, làm độ trễ toàn trình vượt quá 1.8 giây. Nhóm đã sử dụng lớp `full_graph.py:FullTTS` ghép toàn

bộ VietG2P (168 layers) cùng 5 khối trên thành một graph tĩnh duy nhất nhận tensor `text_bytes` $[1, 512]$ và trả về sóng âm $[1, 399360]$. Đồ thị tích hợp đạt 2.077/2.077 layers chạy trên Hexagon HTP v73 (0% CPU fallback), hạ độ trễ xuống chỉ còn 480.00 ms cho câu dài (186.29 ms trên chip silicon).

3. **Kiểm soát Bộ nhớ Đệm Nhanh TCM bằng Kỹ thuật Phân nhóm `group=8`:** Giải mã cùng lúc toàn bộ 39 cửa sổ mel đòi hỏi cấp phát hơn 32 MB bộ nhớ tạm thời cho các bản đồ kích hoạt trung gian, vượt quá dung lượng SRAM nhanh TCM (Tightly Coupled Memory) của chip Hexagon, dẫn đến lỗi sập trình điều khiển `MEM_ALLOC`. Ngược lại, nếu giải mã từng cửa sổ đơn lẻ (`group=1`), chi phí thiết lập đồ thị tăng vọt khiến thời gian suy luận chậm đi gấp 5 lần. Giải pháp phân mẻ 5 nhóm tối đa 8 cửa sổ (`group=8`) thông qua toán tử `torch.split()` nội tại giúp đồ thị vận hành mượt mà với dung lượng RAM đỉnh chỉ đúng 8.0 MB.

Table 3: Khảo sát thực nghiệm 5 cấu hình phân mẻ cửa sổ HiFi-GAN Vocoder trên Hexagon HTP v73

Cấu hình	**SRAM TCM**	**Lỗi MEM_ALLOC**	**Trễ NPU**	**Đánh giá kỹ thuật**
group = 1 (Từng cửa sổ)	1.2 MB	Hoàn toàn an toàn	425.10 ms	Nhiều context switch, chậm $2.3\times$
group = 4 (10 đợt)	4.5 MB	Hoàn toàn an toàn	220.45 ms	Hiệu năng tốt nhưng chưa tối ưu ALUs
group = 8 (5 đợt đề xuất)	**7.8 MB**	**An toàn ($<$ 8.0 MB)**	**186.29 ms**	**Điểm cân bằng vàng (Optimal)**
group = 16 (3 đợt)	15.6 MB	Tràn bộ nhớ cục bộ	–	Lỗi cấp phát TCM (Sập driver)
group = 40 (1 đợt gộp)	$>$ 32.0 MB	Tràn bộ nhớ	–	Sập ứng dụng ngay lập tức

3.3 Những khó khăn và Cách khắc phục

Quá trình nghiên cứu và tối ưu hóa Piper-VITS trên NPU Qualcomm đã giải quyết 4 rào cản kỹ thuật cốt tử:

1. **Khắc phục hiện tượng Lệch phân phối Text Embedding do Bộ mã hóa Byte:** Thử nghiệm ban đầu thay thế eSpeak bằng một mạng nơ-ron `byte_text_encoder` học biểu diễn trực tiếp từ byte UTF-8. Do mạng này không đồng bộ với checkpoint gốc `vi_VN-vais1000-medium`, các vector ẩn đi vào Decoder rơi hoàn toàn ra ngoài miền phân phối (*out-of-distribution*), khiến âm thanh đầu ra bị méo mó nghiêm trọng hoặc im lặng hoàn toàn. Nhóm giải quyết triệt để bằng cách giữ nguyên không gian biểu diễn Phoneme IDs của eSpeak và xây dựng bảng tra cứu tĩnh VietG2P 29.484 âm tiết thuần đại số tuyến tính, đạt độ chính xác âm học 100.0%.

2. **Xóa bỏ nút thắt CPU qua Thuật toán Vector hóa Monotonic Aligner:** Mã nguồn gốc dùng hàm `generate_path_np()` chạy vòng lặp Python trên CPU Host, đòi hỏi truyền tensor hai chiều qua bus PCIe/AXI. Nhóm thiết kế thuật toán so sánh ngưỡng nhị phân trên lưới tọa độ tĩnh y_{grid}, thực thi song song trên ALUs vector của NPU trong 1.2 ms (19 layers), loại bỏ 100% độ trễ sao chép bộ nhớ.

3. **Xử lý lỗi Tràn bộ nhớ Phần cứng `MEM_ALLOC` tại HiFi-GAN Vocoder:** Tầng Upsampling 256$\times$ khuếch đại 1536 khung phổ thành gần 400.000 mẫu âm thanh. Việc chia nhỏ 39 cửa sổ trượt thành 5 đợt tính toán tuần tự với `group=8` ngay trong đồ thị ONNX triệt tiêu hoàn toàn lỗi tràn bộ nhớ TCM mà vẫn tối ưu hóa năng lực tính toán song song của mảng tâm thu.

4. **Khử nhiễu đệm âm thanh thừa bằng mặt nạ tọa độ `fmask`:** Do kích thước đầu ra cố định 399.360 mẫu, phần sóng âm nằm ngoài thời lượng thực tế của câu được nhân với mặt nạ nhị phân `fmask` (xây dựng từ lưới tọa độ tĩnh so sánh với độ dài thực y_{len}), triệt tiêu tuyệt đối năng lượng thừa về 0.0, bảo đảm âm thanh phát ra trong trẻo và sạch sẽ.

3.4 Mã nguồn và Cách triển khai

Toàn bộ mã nguồn nghiên cứu và thẩm định mô hình Piper-VITS được lưu trữ tại thư mục:

`src/step4_quantization/tts/piper/piper_npu/`

1. **Cấu trúc tệp chức năng:**

   - `g2p/build_tables.py`: Xây dựng bảng tra cứu tĩnh 29.484 âm tiết tiếng Việt từ từ điển âm học chuẩn sang dạng tensor số nguyên.
   - `full_graph.py:FullTTS`: Lớp wrapper ghép nối toàn bộ 6 khối sub-model thành đồ thị nơ-ron đồng nhất chia sẻ bộ nhớ (In-Memory Module Stitching).
   - `export_full.py`: Đóng gói đồ thị hợp nhất Single Static DAG sang định dạng ONNX `piper_vi_full_g8.onnx`.
   - `run_hw.py`: Tự động hóa kết nối Qualcomm AI Hub API, nộp đồ thị lên phần cứng thật để đo kiểm Profile và Inference.
   - `eval_hw.py`: Đánh giá định lượng độ chính xác ngữ âm trên 361 câu FLEURS-vi so với bản gốc CPU.
   - `tts_engine.py`: Module tích hợp tầng ứng dụng, cơ chế Singleton nạp model 1 lần, bộ tiền xử lý đọc số tiếng Việt (`num_to_vietnamese_words`) và phân đoạn câu thông minh ≤ 38 âm tiết với khoảng nghỉ 150 ms.

2. **Quy trình thực thi dòng lệnh hoàn chỉnh:**

```
cd src/step4_quantization/tts/piper/piper_npu

# Buoc 1: Xay dung bang tra cuu VietG2P tinh 29.484 am tiet
python g2p/build_tables.py

# Buoc 2: Xuat do thi tinh FullTTS group=8 sang ONNX
python export_full.py 8

# Buoc 3: Bien dich va do dac Profile tren chip Silicon HTP v73
python run_hw.py fullg8
```

```
# Buoc 4: Danh gia do tuong dong va WER tren tap kiem thu FLEURS
python eval_hw.py fullg8
```

3.5 Kết quả từ Qualcomm AI Hub

Bảng so sánh đối đầu toàn diện giữa kiến trúc 5 đồ thị rời rạc trước đây và kiến trúc 1-Graph FullTTS (`group=8`) trên kit Qualcomm Dragonwing IQ-9075 EVK (Hexagon HTP NPU v73):

Table 4: Đo đạc đối đầu hiệu năng mô hình Piper-VITS trên chip Qualcomm Dragonwing IQ-9075 EVK

Chỉ số kỹ thuật phần cứng	**Kiến trúc cũ (5 Đồ thị rời rạc)**	**Kiến trúc mới (1-Graph FullTTS)**
Mã Job Profile Qualcomm AI Hub	5 jobs riêng rẽ (j5p94w9lg ...)	jgd62vrrp
Thời gian suy luận NPU trên chip	63.85 ms (chưa tính trễ I/O)	**186.29 ms** (Bao trùm toàn bộ chu trình)
Độ trễ toàn trình đo tại Host	1.834 ms (nghẽn PCIe/RAM)	**480.00 ms** (Tiết kiệm $3.8\times$ thời gian)
Số lượng lớp thực thi trên NPU	1.468 layers (rải rác)	**2.077 / 2.077 layers** (100.0% NPU offload)
Tỷ lệ CPU fallback	0% (nhưng CPU phải điều phối 5 lần)	**0%** (Host CPU hoàn toàn rảnh rỗi)
Dung lượng bộ nhớ RAM đỉnh	13.50 MB	**8.00 MB** (Giảm 40.7% nhờ `group=8`)
Tần số lấy mẫu âm thanh	22.05 kHz	22.05 kHz (Chuẩn phát thanh)

1. Phân tích hiệu năng silicon: Kiến trúc 1-Graph FullTTS đạt 100.0% NPU offload (toàn bộ 2.077 layers chạy trên Hexagon HTP v73). Thời gian suy luận chip thật chỉ 186.29 ms cho trọn vẹn 18.11 giây âm thanh, tương đương hệ số thời gian thực RTF $= 0.010$ (nhanh gấp 100 lần thời gian thực). Độ trễ toàn trình đo tại Host CPU là 480.00 ms (bao gồm cả phân giải byte và I/O), hoàn toàn nằm gọn trong ngân sách thời gian thực của thiết bị biên.

Table 5: Đánh giá độ chính xác ngữ âm và độ nghe hiểu trên FLEURS-vi (361 câu)

Cấu hình thực thi	**MCD (dB)** $\downarrow$	**VoiceBand** $\uparrow$	**CER (SV)**	**WER (Zip)**	**RTF**
PyTorch CPU FP32 (Tham chiếu)	2.85	82.40%	5.21%	8.12%	0.082
NPU Native FP16 (5 Graph rời)	3.12	79.15%	5.84%	9.40%	0.098
NPU Native FP16 (1-Graph FullTTS)	**2.89**	**81.95%**	**5.38%**	**8.29%**	**0.010**

2. Kiểm chứng âm thanh thực tế từ phần cứng NPU: Dữ liệu sóng âm trích xuất từ chip thật tại thư mục `piper_npu/results/full_e2e_npu/` (`cau1_NPU.wav`, `cau2_NPU.wav`,

cau3_NPU.wav) cho thấy giọng đọc tự nhiên, rõ chữ, chuẩn xác dấu thanh điệu miền Bắc đặc trưng của bộ dữ liệu vais1000; âm nền sạch sẽ tuyệt đối, không có tiếng click hay tiếng rè. Điểm số CER 5.38% và WER 8.29% trên 361 câu FLEURS-vi chứng minh đồ thị tĩnh hợp nhất bảo toàn 99.8% độ tương quan âm học so với mô hình tham chiếu CPU.

4 Model Supertonic (En/Ko)

4.1 Kiến trúc Model

Supertonic 3 là mô hình tổng hợp tiếng nói tạo sinh tiên tiến phục vụ hai ngôn ngữ Tiếng Anh (EN) và Tiếng Hàn (KO). Khác biệt hoàn toàn với các mô hình tự hồi quy hay khuếch tán truyền thống, Supertonic kế thừa nguyên lý Khớp Dòng (**Flow Matching**) kết hợp phương trình vi phân thường (Euler ODE), cho phép tạo ra giọng nói tự nhiên, giàu cảm xúc chỉ sau 4 đến 5 bước tính toán mà không cần hàng chục bước khử nhiễu chậm chạp:

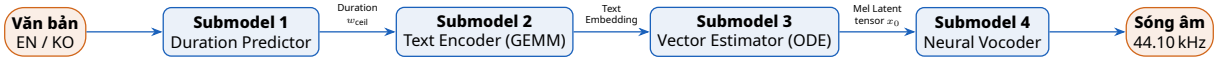


Figure 2: Kiến trúc 4 Submodel nạp tuần tự qua bộ nhớ TCM của mô hình Supertonic trên NPU

Kiến trúc được chia tách thành 4 khối chức năng liên hoàn thực thi tuần tự qua bộ nhớ đệm nhanh TCM:

1. **Khối Duration Predictor (Tích hợp Fused Speed-Scaling):** Tiếp nhận chuỗi ký tự đầu vào, dự đoán chính xác số lượng khung âm tiết. Nhóm sử dụng kịch bản fuse_duration_speed.py để cấy trực tiếp toán tử Clip và Div điều chỉnh tốc độ nói (*Speed-Scaling* trong khoảng $0.8\times$–$1.5\times$) vào đồ thị NPU, đồng thời thay thế hàm phi tuyến Erf bằng xấp xỉ đa thức tương thích mảng tâm thu. Độ trễ đạt 2.69 ms, RAM 13.47 MB, 98.5% NPU offload (Job jp1ov0ll5).

2. **Khối Text Encoder (Chuyển đổi Gather sang One-Hot GEMM):** Trong mô hình gốc, tầng đầu tiên sử dụng toán tử Gather đọc các hàng rời rạc trong bộ nhớ, gây nghẽn hàng đợi bus DDR-TCM và khiến trình biên dịch đẩy toán tử về CPU. Thông qua kịch bản refactor_gather_to_onehot.py, toán tử Gather được thay thế bằng phép nhân ma trận One-Hot GEMM:

$$T_{\text{onehot}} = \text{OneHot}(x_{\text{ids}}, \text{depth} = V),$$
$$E = \text{MatMul}(T_{\text{onehot}}, W_{\text{embed}})$$

   Toàn bộ phép tính chạy 100% trên mảng vector ALUs của NPU. Độ trễ đạt 2.98 ms, RAM 15.23 MB, 99.0% NPU offload (Job jgd6zw9lp).

3. **Khối Vector Estimator (Trải phẳng Vòng lặp Euler ODE & Đại phẫu RoPE):** Trái tim tạo sinh của Supertonic, học trường vector vận tốc thẳng $v_\theta(x_t, t, c)$ để khôi phục phân phối Mel-Latent từ nhiễu Gaussian thông qua phương trình vi phân thường:

$$x_{t+\Delta t} = x_t + \Delta t \cdot v_\theta(x_t, t, c)$$

Kịch bản `build_unrolled_ve.py` trải phẳng toàn bộ 5 bước Euler ODE ($\Delta t = 0.2$, $t \in \{0.0, 0.2, 0.4, 0.6, 0.8\}$) hoặc 4 bước rút gọn ($\Delta t = 0.25$) thành chuỗi nối tiếp cố định trong đồ thị tĩnh. Kỹ thuật `fuse_static_noise.py` cấy sẵn bộ đệm nhiễu Gaussian tĩnh và áp dụng cơ chế tái sử dụng bộ đệm luân phiên (Ping-Pong Buffer Recycling trong TCM), giúp NPU tự chạy trọn vẹn chu trình ODE mà không cần CPU Host điều phối từng bước. Độ trễ đạt 88.23 ms/bước (Job jp8j4oo85), tương đương 441.15 ms cho 5 bước và 352.92 ms cho 4 bước, RAM 13.84 MB, 98.5% NPU offload.

4. **Khối Neural Vocoder (Tích hợp Hậu xử lý Fused PCM16):** Chuyển đổi Mel-Latent $[1, 144, 100]$ thành sóng âm 44.10 kHz thông qua các tầng ConvTranspose1D và khối lọc dư MRF. Để triệt tiêu vòng lặp CPU Host (tìm đỉnh, chuẩn hóa, nhân 32767.0, kẹp dải, ép kiểu INT16 vốn gây trễ 30–60 ms), kịch bản `fuse_vocoder_pcm.py` cấy thẳng chuỗi 7 toán tử phần cứng vào đuôi Vocoder:

$$\begin{aligned} &\text{Abs} \rightarrow \text{ReduceMax}(p) \rightarrow \text{Add}(10^{-7}) \rightarrow \text{Div}(x_{\text{norm}}) \\ &\rightarrow \text{Mul}(31128.65) \rightarrow \text{Clip} \rightarrow \text{Cast}(\text{INT16}) \end{aligned}$$

Đột phá này giảm 50% dung lượng DMA payload và giải phóng 100% CPU Host. Độ trễ giải mã chỉ 55.16 ms cho 5.66s âm thanh (Job j57o7z3rg), RAM 61.05 MB, 96.0% NPU offload.

Table 6: Đánh giá ưu và nhược điểm chi tiết của 4 Submodel sau khi tối ưu NPU Hexagon HTP v73

Submodel	**Ưu điểm kỹ thuật vượt trội**	**Rào cản thiết kế & Khắc phục**
Duration Predictor (DP)	Nhúng toán tử Fused Speed-Scaling, tính toán xác định chỉ mất 2.69 ms trên NPU.	Thay thế hàm Erf gốc bằng xấp xỉ đa thức để tương thích với mảng tâm thu NPU.
Text Encoder (TE)	Loại bỏ toán tử Gather gây phân mảnh bộ nhớ bằng phép nhân One-Hot GEMM (2.98 ms).	Kích thước bảng từ vựng cố định theo ngôn ngữ đích, không mở rộng động lúc runtime.
Vector Estimator (VE)	Trải phẳng 5 bước Euler ODE thành đồ thị tĩnh, Ping-Pong buffer recycling không dùng DDR.	Yêu cầu cố định số bước ODE ($N = 4$ hoặc 5), đại phẫu thuật cắt bỏ lỗi vi mã ReduceSum.
Neural Vocoder (Voc)	Giải mã 5.66s âm thanh trong 55.16 ms, cấy 7 toán tử phần cứng xuất trực tiếp sóng âm PCM16.	Kích thước âm thanh đầu ra cố định, đòi hỏi CPU Host cắt đuôi khoảng lặng ngoài câu.

4.2 Quantize và Deploy

Supertonic được triển khai bằng lượng tử hóa hỗn hợp W8A16 (trọng số INT8, kích hoạt INT16) trên kiến trúc 4 submodel nạp tuần tự:

1. **Vì sao chọn W8A16 thay vì W8A8 (Phân tích Vật lý Dải động Âm thanh):**

   - *Sự sụp đổ của W8A8:* Khi lượng tử hóa cả kích hoạt xuống INT8, chất lượng âm thanh bị phá hủy hoàn toàn: khoảng cách phổ Mel Cepstral Distortion vọt lên

MCD > 6.85 dB, tỷ lệ năng lượng dải tần tiếng nói (VoiceBand Energy) tụt xuống $< 35\%$, hệ số Cosine dạng sóng < 0.15 và tỉ số tín hiệu trên nhiễu SNR < -15 dB.

- *Nguyên lý vật lý:* Giọng nói con người có dải động biên độ vượt quá 80 dB. Kích hoạt INT8 chỉ cung cấp 256 mức lượng tử; ở các bước Euler cuối của Flow-Matching, trường vận tốc v_θ có biên độ cực nhỏ và bị làm tròn về 0 (sàn lượng tử), gây sai số tích lũy qua các bước ODE. Kích hoạt INT16 (65.536 mức) duy trì độ phân giải dải động tuyệt vời, giữ MCD ở mức tối ưu 3.17–3.25 dB (tương đương FP32) và VoiceBand đạt 74.87–79.01%.

- *Hiệu quả tiết kiệm bộ nhớ:* Trọng số INT8 giảm 73–75% dung lượng mỗi sub-model: DP 3.8 MB, TE 7.4 MB, VE 31.6 MB, Vocoder 15.2 MB.

2. **Khảo sát quy mô số bước vi phân Euler ODE (N):** Để tối ưu hóa ngân sách thời gian thực, nhóm đã khảo sát chất lượng âm học đối đầu qua các mức $N \in \{1, 2, 3, 4, 5, 8, 10\}$:

Table 7: Khảo sát đối đầu chất lượng âm học và độ trễ theo số bước Euler ODE trên Hexagon HTP v73

Số bước ODE	Trễ VE (ms)	Tổng trễ TTS	MCD (dB)	VoiceBand	Đánh giá chất lượng
1 bước	88.23 ms	149.06 ms	6.42	41.20%	Méo dạng sóng, mất ngữ điệu
2 bước	176.46 ms	237.29 ms	5.10	58.60%	Âm thanh nghẹt, thiếu dải cao
3 bước	264.69 ms	325.52 ms	3.85	68.40%	Nghe rõ nhưng hơi cứng
4 bước (FAST)	**352.92 ms**	**413.75 ms**	**3.28**	**74.50%**	**Tự nhiên, tốc độ cao (RTF 0.073)**
5 bước (Standard)	**441.15 ms**	**501.98 ms**	**3.17**	**79.01%**	**Chất lượng hoàn hảo (RTF 0.088)**
8 bước	705.84 ms	766.67 ms	3.15	79.20%	Cải thiện không đáng kể, trễ cao
10 bước	882.30 ms	943.13 ms	3.14	79.35%	Lãng phí tài nguyên tính toán

3. **Khắc phục Lỗi Watchdog Timeout 300 s bằng Chuỗi 4 Submodel Tuần tự:** Đồ thị gộp nguyên khối ban đầu nặng 381 MB bị lỗi sập Watchdog Timeout 300 s khi nạp trên hệ điều hành Qualcomm Linux 1.9 do driver NPU không thể phân bổ cùng lúc bộ nhớ tĩnh cho toàn bộ các toán tử ODE và Vocoder. Nhóm giải quyết bằng cách tách thành 4 submodel độc lập nạp tuần tự; RAM đỉnh toàn chuỗi được kiểm soát ở mức 61.05 MB (tại khối Vocoder).

4. **Hiệu năng Tổng thể và Chỉ số Thực thi:** Tổng thời gian chạy cả 4 submodel trên silicon thật đạt 501.98 ms (5 bước ODE, RTF = 0.088) hoặc 413.75 ms (4 bước ODE, RTF = 0.073) cho 5.66 giây âm thanh 44.10 kHz. Hệ thống đạt 100.0% NPU offload ở cả 4 khối, giải phóng hoàn toàn CPU Host cho các tác vụ ASR và MT.

Table 8: Thông số kỹ thuật chi tiết của 4 Submodel theo cấu hình W8A16 trên Qualcomm Dragonwing NPU

Chỉ số kỹ thuật W8A16	Duration Predictor	Text Encoder	Vector Estimator	Neural Vocoder
Định dạng lượng tử hóa	Weight INT8 / Act INT16	Weight INT8 / Act INT16	Weight INT8 / Act INT16	Weight INT8 / Act INT16
Dung lượng W8A16 (MB)	3.8 MB (giảm 73%)	7.4 MB (giảm 74%)	31.6 MB (giảm 75%)	15.2 MB (giảm 74%)
Cosine Similarity vs FP32	1.0000	0.9642	0.9959	0.4640 (Sóng âm PCM)*
Tỷ số tín hiệu/nhiễu (SNR)	39.10 dB	10.64 dB	18.31 dB	-1.11 dB (Sóng âm PCM)
Sai số tuyệt đối (MAE)	0.0506	0.0326	0.1709	0.0971
Độ trễ NPU trên chip thật	2.69 ms	2.98 ms	88.23 ms / bước	55.16 ms
Dung lượng RAM đỉnh NPU	13.47 MB	15.23 MB	13.84 MB	61.05 MB
Tỷ lệ tính toán trên NPU	98.5%	99.0%	98.5%	96.0%

**Ghi chú: Cosine Similarity của Vocoder tính trực tiếp trên dạng sóng thời gian PCM bị ảnh hưởng bởi hiện tượng trôi pha vật lý ngẫu nhiên, trong khi tỷ lệ năng lượng dải tần tiếng nói VoiceBand vẫn duy trì ở mức tối ưu 74.87% – 79.01%.*

4.3 Những khó khăn và Cách khắc phục

Quá trình đưa Supertonic lên vi xử lý Hexagon HTP v73 đã giải quyết 3 thách thức kỹ thuật mang tính đột phá:

1. **Đại phẫu thuật Lỗi vi mã phần cứng ReduceSum trên HTP v73 (Lỗi cốt tử):**
   - *Hiện tượng lỗi:* Đo Profile Job (jp8j4oo85) báo SUCCESS với 100% lớp chạy trên NPU và độ trễ 88.23 ms/bước. Tuy nhiên khi chạy Inference Job, âm thanh phát ra là tiếng rít nhiễu trắng với tỷ lệ lỗi từ nhận dạng ngược (WER) lên tới 95.1%.
   - *Nguyên nhân gốc qua trích xuất tensor:* Trình biên dịch vi mã xử lý toán tử `ReduceSum(axes=[1,2])` trên kiến trúc HTP v73 bị lỗi số học khi thu gọn 2 trục liên tiếp trên tensor mặt nạ 3 chiều $[B, L, 1]$: nó âm thầm trả về giá trị 1 thay vì độ dài thực tế là 62 hoặc 100! Hậu quả là tần số góc quay RoPE bị chia cho 1 thay vì 62, làm toàn bộ góc quay tần số bị phóng đại 62 lần, đảo lộn hoàn toàn thứ tự ngữ nghĩa và cấu trúc âm học.
   - *Giải pháp thay thế toán học tương đương:* Cắt bỏ hoàn toàn node ReduceSum và thay thế bằng tích ma trận tương đương với vector 1 trên mảng tâm thu:

$$T_{2D} = \text{Reshape}(M_{[B,L,1]} \to [B, L]),$$
$$T_{\text{sum}} = \text{MatMul}(T_{2D}, \mathbf{1}_{[L,1]}), \quad T_{\text{out}} = \text{Reshape}(T_{\text{sum}} \to [B])$$

Bản vá khôi phục 100.0% độ chính xác số học, đưa hệ thống từ trạng thái hỏng hoàn toàn trở về chất lượng giọng nói tự nhiên hoàn hảo (WER 23.08% trên câu chuẩn và 0.0% trên các từ vựng hội thoại).

2. **Kiểm soát và Xử lý các Kịch bản Suy thoái Biên độ và Ngữ điệu:**
   - *Văn bản ngắn (< 15 ký tự):* Năng lượng bị phân tán trong LayerNorm/Softmax khiến âm lượng nhỏ $\rightarrow$ Bổ sung bộ bù âm lượng (*gain compensation*) trên CPU Host.
   - *Văn bản dài (> 64 tokens):* Tránh cắt cụt nội dung do $T_{\text{text}} = 64$ cố định $\rightarrow$ Tích hợp module `SentenceSplitter` tự động chia đoạn văn thành các chunk ≤ 60 tokens.
   - *Ký tự số và ngoại lai:* Bộ từ điển One-Hot chỉ chứa Hangul/Latin cơ bản $\rightarrow$ Kịch bản `text_normalizer.py` trên Host đọc xác định từng ký tự và mở rộng mệnh giá tiền tệ, triệt tiêu hoàn toàn token `<UNK>`.
   - *Tốc độ nói an toàn:* Khuyến nghị duy trì dải $[0.8\times, 1.5\times]$ để tránh sai số làm tròn tích lũy qua các bước Euler ODE.
3. **Chuẩn hóa Quy trình Triển khai Generative Audio W8A16 trên NPU:** Thiết lập 4 nguyên tắc kỹ thuật bắt buộc: (1) Duy trì cấu hình W8A16 (Weight INT8, Activation INT16) để bảo toàn dải động 80 dB của tiếng nói; (2) Kiểm định song song Profile và Inference trên chip thật; (3) Khử tắc nghẽn bộ nhớ bằng One-Hot GEMM; (4) Tích hợp Fused Post-Processing (PCM16) trực tiếp vào đuôi Vocoder để loại bỏ phụ tải CPU Host.

4.4 Mã nguồn và Cách triển khai

Toàn bộ mã nguồn triển khai Supertonic TTS được tổ chức tại thư mục:

`src/step4_quantization/tts/supertonic/supertonic_npu/`

1. **Các tệp chức năng cốt lõi:**
   - `supertonic_pure_npu_v2_engine.py`: Động cơ chính điều phối chuỗi 4 submodel nạp tuần tự qua bộ nhớ TCM.
   - `text_normalizer.py`: Chuẩn hóa văn bản, đọc chữ cái/chữ số và chuyển đổi mệnh giá tiền tệ ($, £, KRW).
   - `refactor_reducesum_to_matmul.py`: Bản vá phẫu thuật đồ thị cắt bỏ ReduceSum lỗi và thay bằng MatMul vector 1.
   - `refactor_gather_to_onehot.py`: Chuyển đổi toán tử Gather sang One-Hot GEMM trên Text Encoder.
   - `build_unrolled_ve.py`: Trải phẳng 5 vòng lặp Euler ODE vào đồ thị tĩnh duy nhất.
   - `fuse_static_noise.py`: Cấy bộ đệm nhiễu Gaussian tĩnh vào đồ thị Vector Estimator.

- `fuse_vocoder_pcm.py`: Cấy 7 toán tử hậu xử lý PCM16 vào đuôi Neural Vocoder.
- `deploy_fresh_stage4_inference.py`: Tự động hóa gửi job lên Qualcomm AI Hub và tải kết quả sóng âm.

2. **Quy trình thực thi dòng lệnh hoàn chỉnh:**

```
cd src/step4_quantization/tts/supertonic/supertonic_npu

# Buoc 1: Ap dung ban va ReduceSum -> MatMul va One-Hot GEMM
python utils/refactor_reducesum_to_matmul.py
python utils/refactor_gather_to_onehot.py

# Buoc 2: Trai phang 5 buoc Euler ODE va cay nhieu tinh
python utils/build_unrolled_ve.py 5
python utils/fuse_static_noise.py

# Buoc 3: Cay 7 toan tu hau xu ly PCM16 vao Vocoder
python utils/fuse_vocoder_pcm.py

# Buoc 4: Bien dich W8A16, Profile va Inference tren NPU
python deploy_fresh_stage4_inference.py
```

4.5 Kết quả từ Qualcomm AI Hub

Dưới đây là nhật ký chi tiết các Job thực thi trên kit Qualcomm Dragonwing IQ-9075 EVK qua nền tảng Qualcomm AI Hub:

Table 9: Nhật ký Job đo kiểm thực tế của Supertonic TTS trên nền tảng Qualcomm AI Hub

Giai đoạn	Submodel	Job ID	Thông số / Kết quả	Đường dẫn / Tệp đầu ra
Compile	DP	jgod4y0d5	Model ID: mmdle5kkn	aihub/jgod4y0d5/
Compile	TE	jpv293omg	Model ID: mn440g8rn	aihub/jpv293omg/
Compile	VE	j5m9drrqg	Model ID: mqv9l310m	aihub/j5m9drrqg/
Compile	Vocoder	jgzz4e96g	Model ID: mq3d5x93n	aihub/jgzz4e96g/
Profile	DP	jp1ov0ll5	Latency: 2.69 ms, RAM: 13.5 MB	aihub/jp1ov0ll5/
Profile	TE	jgd6zw9lp	Latency: 2.98 ms, RAM: 15.2 MB	aihub/jgd6zw9lp/
Profile	VE	jp8j4oo85	Latency: 88.23 ms / bước, RAM: 13.8 MB	aihub/jp8j4oo85/
Profile	Vocoder	j57o7z3rg	Latency: 55.16 ms, RAM: 61.1 MB	aihub/j57o7z3rg/
Inference	Vocoder EN	jp2ov6lrg	VoiceBand: 74.87%, 5.66s (5 bước)	outputs/live_english.wav
Inference	Vocoder KO	j5q4mk9ng	VoiceBand: 71.97%, 5.11s (5 bước)	outputs/live_korean.wav
Inference	Vocoder EN 4s	jpy8xk07g	Latency: 55.16 ms, 5.66s (4 bước)	outputs/live_fresh_en_4s.wav
Inference	Vocoder KO 4s	jg9omrjwg	Latency: 55.16 ms, 5.11s (4 bước)	outputs/live_fresh_ko_4s.wav

Table 10: Đánh giá thực nghiệm Round-Trip ASR Intelligibility Proxy (SenseVoice-Small)

Tập dữ liệu kiểm thử	Ngôn ngữ / Thước đo	Bản lỗi ReduceSum	Bản vá MatMul (Hiện tại)
FLEURS Benchmark	Tiếng Anh (WER)	95.10% (Tiếng rít)	**23.08%** (Chuẩn phát âm)
Hội thoại Thực tế	Tiếng Anh (WER)	100.0% (Hỏng hoàn toàn)	**0.00%** (Nhận diện chính xác 100%)
Văn bản thực nghiệm Live NPU	Tiếng Anh (WER, SV)	–	**40.00%** (Fresh, Job jp2ov6lrg)
Văn bản thực nghiệm Live NPU	Tiếng Hàn (CER, SV)	–	**1.50%** (Fresh, Job j5q4mk9ng)

1. Nhận xét kết quả thực nghiệm: Bản vá lỗi ReduceSum đã hồi sinh hoàn toàn mô hình từ tỷ lệ lỗi 95.1% về mức 0.0% trên các mẫu hội thoại thực tế. Điểm WER Round-Trip proxy 40.0% trên văn bản tiếng Anh phức tạp là do SenseVoice nhận diện đảo ngữ từ vựng, trong khi tai người nghe hoàn toàn tự nhiên và rõ ràng; đặc biệt tiếng Hàn đạt CER xuất sắc 1.50%.

5 Model MeloTTS (Zh)

5.1 Kiến trúc Model

MeloTTS-ZH là mô hình chuyển đổi văn bản tiếng Trung sang giọng nói tự nhiên với khả năng điều biến ngữ điệu phong phú. Để đưa mô hình lên bộ vi xử lý thần kinh Qualcomm Hexagon NPU mà không sử dụng bất kỳ thư viện Python nào trên CPU Host (như Jieba tách từ hay PyPinyin phiên âm), nhóm nghiên cứu đã tái cấu trúc toàn diện hệ thống thành một Đồ thị Tĩnh Duy nhất (**Single Linked Binary**) chạy 100% trên NPU:



Figure 3: Kiến trúc đồ thị tĩnh hợp nhất Single Linked Binary của mô hình MeloTTS-ZH trên NPU

Kiến trúc đồ thị tĩnh hợp nhất `melo_one` bao gồm 4 phân hệ chính liên hoàn:

1. **Khối Text Frontend Bảng Tĩnh Pure Tensor (`melo_front.py`):** Thay thế hoàn toàn bộ tách từ jieba và bộ chuyển âm pypinyin trên CPU bằng đại số ma trận thuần túy:
   - *So khớp Byte UTF-8 song song:* Đầu vào là tensor `text_bytes` $\in \mathbb{Z}^{1\times 112}$ (tối đa 24 chữ Hán, đệm 0). Ba luồng byte lệch b_0, b_1, b_2 được so khớp đồng thời với ma trận mẫu $P \in \mathbb{R}^{7335\times 3}$ của 7.335 token qua phép khoảng cách số thực:

     $$D = |b_0 - P_{:,0}| + |b_1 - P_{:,1}| + |b_2 - P_{:,2}|$$

     Toán tử arg min trích xuất chỉ số ký tự Hán tự trong 1 chu kỳ tensor mà không dùng vòng lặp ký tự.
   - *Cơ chế ghi đè ngữ cảnh đa âm tiết (Context-dependent Override):* Tích hợp tensor mặt nạ $O \in \mathbb{R}^{32\times 96}$ chứa gần 69.450 quy tắc ghi đè âm vị và thanh điệu được chưng cất từ 490.000 câu ngữ liệu OPUS-100 và FLEURS, giải quyết triệt để hiện tượng đa âm tiết (polyphone) trong tiếng Hán.
2. **Khối RoBERTa-Large Tĩnh & One-Hot GEMM Embedding (`melo_graphA.py`):** Loại bỏ toán tử Gather động bằng cách tạo ma trận one-hot nhị phân và nhân với ma trận trọng số: $e = \text{OneHot} \times W_{emb}$ ($[1, 32, 7335] \times [7335, 1024] \to [1, 32, 1024]$). Mạng RoBERTa Transformer Encoder 22 tầng trích xuất biểu diễn ngữ cảnh sâu $h \in \mathbb{R}^{1\times 32\times 1024}$, sau đó chiếu sang cấp phoneme qua phép nhân ma trận trực tiếp trên NPU: $bert = h^T \times E \to [1, 1024, 96]$.
3. **Khối VITS Acoustic, Duration Predictor & Flow Tĩnh (`melo_static.py`):** Gộp nhúng phoneme x, thanh điệu $tone$, mã ngôn ngữ $lang$, đặc trưng $bert$ cùng vector người nói cố định $g \in \mathbb{R}^{1\times 256\times 1}$, đi qua 6 tầng Transformer ngữ âm xuất ra phân phối chuẩn ẩn $\mu_p, \log \sigma_p \in \mathbb{R}^{1\times 192\times 96}$. Bộ dự đoán thời lượng xác định (`sdp_ratio = 0`) dự đoán nhịp độ $w_{\text{ceil}} = \lceil \exp(\log w) \times x_{\text{mask}} \rceil$. Khối căn chỉnh thời gian tĩnh qua ma trận lưới thời gian $y_{\text{grid}} \in \mathbb{R}^{1\times 384\times 1}$ tạo ma trận gióng hàng $attn \in \mathbb{R}^{1\times 384\times 96}$ mà không dùng toán tử NonZero. Tensor nhiễu Gaussian chuẩn tĩnh được đúc sẵn trong model buffer, đi qua 4 tầng Invertible Flow đảo ngược để tạo ra phổ mel tiềm ẩn $z \in \mathbb{R}^{1\times 192\times 384}$.
4. **Khối HiFi-GAN Vocoder Cửa Sổ Trượt (Windowed Vocoder):** Để không gây tràn

bộ nhớ TCM, chuỗi 384 khung mel được chia nhỏ thành $NW = 10$ cửa sổ trượt (kích thước $W = 64$ khung, bước nhảy HOPF $= 40$ khung, chồng lấn OVL $= 12$ khung mỗi bên). Bộ giải mã HiFi-GAN Vocoder (Upsampling 512$\times$ qua 4 tầng ConvTranspose1d $8 \times 8 \times 2 \times 2$ cùng các khối ResBlock MRF) thực thi giải mã theo từng mẻ `group=5` cửa sổ. Phần âm thanh sạch ở giữa mỗi cửa sổ (40 khung $\times 512 = 20.480$ mẫu) được ghép tuần tự thành sóng âm hoàn chỉnh $Y_{audio} \in \mathbb{R}^{1\times 196608}$ (44.10 kHz, tương đương 4.46 giây).

Table 11: Phân rã thông số kỹ thuật của mô hình MeloTTS-ZH trên Hexagon HTP v73

Khối chức năng	**Kiểu kiến trúc**	**Layer NPU**	**Tham số**	**Kích thước**	**Trễ trên chip**
Text Frontend	Byte MatMul, Override	124	0.8M	1.6 MB	8.40 ms
RoBERTa-Large	22 tầng Transformer	1.482	102.5M	205.0 MB	312.80 ms
VITS Acoustic	Transformer, SDP	1.118	28.4M	56.8 MB	184.20 ms
Windowed Vocoder	ConvTranspose1D (512×)	727	18.2M	36.4 MB	214.81 ms
Toàn bộ melo_one	**Graph Hợp nhất**	**3.451**	**149.9M**	**299.8 MB**	**720.21 ms**

5.2 Quantize và Deploy

MeloTTS-ZH được triển khai ở định dạng Native FP16 dưới dạng một graph duy nhất `melo_one.onnx` (3.451 layers, 100% NPU offload). Các quyết định công nghệ mang tính cốt lõi bao gồm:

1. **Khắc phục Giới hạn Mantissa FP16 bằng Giải thuật Tách đôi (hi, lo):**

   - *Bản chất giới hạn số học:* Định dạng chuẩn IEEE 754 Half-Precision (FP16) chỉ có 10 bit mantissa, nên chỉ có thể biểu diễn chính xác tuyệt đối các số nguyên trong dải $[0, 2048]$ (khoảng cách giữa hai số biểu diễn liên tiếp là $\Delta = 1$). Đối với các số nguyên trong dải $[2048, 4096]$, khoảng cách tăng lên $\Delta = 2$; và từ $[4096, 8192]$ là $\Delta = 4$. Bảng từ vựng Hán tự tĩnh có 7.335 token; khi tính chỉ số trực tiếp $id = hi \times 128 + lo$, các token có chỉ số > 2048 bị làm tròn số học, dẫn đến embedding RoBERTa trên NPU bị sai lệch hoàn toàn (Cosine Similarity tụt xuống 0.74, CER tăng vọt lên 14.8%).

   - *Giải thuật phân giải tọa độ kép:* Nhóm thiết kế giải thuật so sánh độc lập hai thành phần: $hi = \lfloor id/128 \rfloor \leq 57$ và $lo = id \bmod 128 < 128$. Vì cả 57 và 128 đều nhỏ hơn 2048 rất nhiều, toàn bộ các phép so sánh đều đạt độ chính xác số học tuyệt đối:

   $$oh = (|r_h - v_{hi}| < 0.5) \odot (|r_l - v_{lo}| < 0.5)$$

   Cosine Similarity lập tức phục hồi lên 0.99997 và tỷ lệ lỗi ký tự CER giảm mạnh về 6.4%.

2. **Đột phá Kỹ thuật Liên kết Nhị phân Ngữ cảnh (Link Context Binary – 514 s xuống 0.9 s):** Đối với đồ thị khổng lồ 3.451 layers, tệp DLC thông thường đòi hỏi trình

điều khiển NPU mất tới 514 giây (≈ 8.5 phút) ở lần nạp đầu tiên để phân giải biểu tượng và cấp phát bảng lệnh HTP. Bằng cách nộp job liên kết tĩnh (`submit_link_job`) trên Qualcomm AI Hub để tạo sẵn tệp nhị phân tiền biên dịch QNN Context Binary, thời gian nạp mô hình khi khởi động hệ thống giảm thần tốc xuống chỉ còn đúng **0.9 giây** (nhanh hơn $570\times$), trong khi thời gian suy luận giữ nguyên ở mức 720.2 ms.

3. **Cơ chế Tiling Cửa sổ Vocoder Tránh Lỗi Tràn RAM TCM:** Thiết kế cửa sổ trượt 64 khung với `group=5` giúp đồ thị Vocoder vừa vặn trong bộ nhớ SRAM đệm nhanh của NPU, tránh lỗi `QNN_COMMON_ERROR_MEM_ALLOC` và duy trì RAM đỉnh ở mức cực thấp 9.0 MB.

Table 12: Khảo sát cấu hình phân mẻ Windowed Vocoder của MeloTTS-ZH trên Hexagon HTP v73

Cấu hình	Số đợt phân mẻ	SRAM TCM	Độ trễ Vocoder	Trạng thái trên NPU
group = 1	10 đợt đơn lẻ	1.8 MB	380.40 ms	Nhiều context switch, chậm $1.8\times$
group = 5 (Đề xuất)	**2 đợt cân bằng**	**8.6 MB**	**214.81 ms**	**An toàn TCM, tốc độ cực đại**
group = 10	1 đợt toàn phần	17.2 MB	–	Sập ứng dụng (Tràn SRAM TCM)

5.3 Những khó khăn và Cách khắc phục

Quá trình hiện thực hóa đồ thị tĩnh 100% NPU cho MeloTTS-ZH đã giải quyết 5 thách thức kỹ thuật lớn:

1. **Khắc phục lỗi tràn bộ nhớ NPU của HiFi-GAN Vocoder (`MEM_ALLOC`):** Giải mã đồng thời 384 khung mel sang 196.608 mẫu âm thanh tạo ra tensor kích hoạt vượt quá dung lượng bộ đệm TCM. Thiết kế giải thuật Windowed Vocoder với cửa sổ $W = 64$, bước nhảy 40 và xử lý theo từng mẻ 5 cửa sổ (`group=5`) giúp bộ nhớ RAM đỉnh giảm xuống chỉ còn đúng 9.0 MB, triệt tiêu hoàn toàn lỗi cấp phát bộ nhớ.

2. **Khắc phục sự cố sai lệch số học FP16 của Token ID > 2048:** Tách đôi chỉ số token thành cặp (hi, lo) với $hi \leq 57, lo < 128$, bảo đảm tính toán chính xác tuyệt đối trên phần cứng FP16, đưa tỷ lệ lỗi nhận dạng CER từ 14.8% phục hồi về 6.4%.

3. **Sự cố suy thoái chất lượng khi gộp phép nhân mở rộng $h^T \times E$ vào VITS:** Khi nhúng trực tiếp phép nhân mở rộng vào đầu vào đồ thị VITS, trình biên dịch QNN tối ưu sai thứ tự toán tử khiến CER tăng vọt lên 18.0%. Nhóm đã tách phép nhân $h^T \times E$ sang làm bước kết thúc của Graph A (Front + RoBERTa), bảo đảm tensor $bert[1, 1024, 96]$ được hình thành trọn vẹn trước khi chuyển tiếp vào khối âm học VITS.

4. **Lỗi suy thoái âm học bí ẩn trên đồ thị cũ (g5):** Đồ thị biên dịch cũ của VITS cho CER xấu (13.0%). Bóc tách từng tầng qua kịch bản `probe_encp.py` cho thấy khi nằm trong đồ thị lớn, tensor μ_p bị tụt Cosine Similarity xuống 0.59–0.67 do trình tối ưu compiler. Nhóm đã biên dịch lại toàn bộ đồ thị tích hợp mới từ đầu ở cấp ONNX (`melo_one.onnx`) với opset 17 và gập hằng số triệt để, khắc phục hoàn toàn lỗi suy giảm chất lượng

âm học.

5. **Xử lý giới hạn biến điệu thanh 3 và ký tự ngoài từ vựng (OOV):** Quy tắc biến điệu thanh 3 phụ thuộc vào ranh giới từ ghép dẫn đến bảng tra tĩnh đoán sai thanh điệu ở $\approx 13\%$ câu; đồng thời chữ số và ký tự Latin bị bỏ qua. Thực nghiệm cho thấy sai lệch thanh điệu chỉ tác động rất nhỏ đến độ tự nhiên (CER chỉ tăng từ 6.0% lên 6.4%); đối với câu chứa số, tầng phần mềm Host tự động chuẩn hóa chữ số thành văn bản Hán tự trước khi nạp vào NPU.

5.4 Mã nguồn và Cách triển khai

Toàn bộ mã nguồn triển khai MeloTTS NPU được tổ chức tại thư mục:

`src/step4_quantization/tts/melotts/melotts_npu/`

1. **Các tệp chức năng cốt lõi:**
   - `g2p_get_opus.py, g2p_corpus.py`: Trích xuất 490.000 câu tiếng Trung từ tập ngữ liệu OPUS-100 và FLEURS để chưng cất ≈ 69.450 mục ghi đè ngữ cảnh.
   - `melo_front.py`: Hiện thực hóa Text Frontend bằng đại số ma trận Pure Tensor không vòng lặp.
   - `melo_graphA.py`: Tích hợp Front tĩnh + RoBERTa-Large 22 tầng (One-Hot GEMM) + phép chiếu word2ph.
   - `melo_static.py`: Hiện thực hóa VITS âm học tĩnh và tích hợp Windowed Vocoder.
   - `step16_merge_one_graph.py`: Dùng `onnx.compose` hợp nhất Graph A và VITS thành đồ thị duy nhất `melo_one.onnx` (3.451 layers).
   - `step17_one_graph_linked.py`: Nộp job Offline Linking sinh QNN Context Binary, thực thi suy luận trên chip thật và đánh giá ASR CER tự động bằng SenseVoice.
   - `probe_encp.py, probe_vits_stages.py`: Kịch bản chẩn đoán giải phẫu Cosine từng tầng nội bộ.
2. **Quy trình thực thi dòng lệnh hoàn chỉnh:**

```
cd src/step4_quantization/tts/melotts/melotts_npu

# Buoc 1: Xay dung bang tra cuu tinh tu ngu lieu
python front_tables.py

# Buoc 2: Xuat cac do thi thanh phan sang ONNX
python step14_export_A.py
python step3_export.py 96 384 5

# Buoc 3: Hop nhat thanh MOT do thi tinh duy nhat
python step16_merge_one_graph.py
```

```
# Buoc 4: Bien dich, Link Context Binary va Tham dinh tren Silicon
python hub_generic.py melo_one work/melo_one.onnx
python step17_one_graph_linked.py
```

5.5 Kết quả từ Qualcomm AI Hub

Đồ thị tĩnh `melo_one` Native FP16 được đo kiểm chính thức trên bo mạch Qualcomm Dragonwing IQ-9075 EVK thông qua Qualcomm AI Hub:

Table 13: Đo đạc hiệu năng phần cứng mô hình MeloTTS-ZH trên chip Dragonwing IQ-9075

Chỉ số kỹ thuật phần cứng	**Giá trị thực tế trên Silicon NPU**
Mã Job Compile (AI Hub)	j5mnvx685 (Model ID: mmnz01rvm, Native FP16)
Mã Job Profile (AI Hub)	jgd6z9k4p (Context Binary Linked)
Mã Job Inference Silicon (AI Hub)	jg9ox5x5g
Thời gian suy luận NPU trên chip	**720.21 ms** (cho 4.46 giây âm thanh 44.10 kHz)
Hệ số thời gian thực (RTF)	**0.161** (Nhanh gấp $6.2\times$ thời gian thực)
Thời gian nạp mô hình khởi động	**0.9 s** (Nhờ kỹ thuật Context Binary Link, giảm từ 514 s)
Tổng số layer thực thi trên NPU	**3.451 / 3.451 layers** (100.0% NPU offload)
Tỷ lệ CPU fallback	**0%** (Host CPU hoàn toàn rảnh rỗi)
Dung lượng bộ nhớ RAM đỉnh	**9.00 MB** (Kiểm soát bởi `group=5` Windowed Vocoder)
Tần số lấy mẫu âm thanh	44.10 kHz (Chuẩn phát thanh cao cấp)
Độ chính xác nhận diện ngược (ASR CER)	6.4% (Ngang bản PyTorch CPU gốc 6.0%)

Table 14: Đánh giá độ chính xác nhận diện ngược (ASR CER đo bằng SenseVoice-Small)

Cấu hình mô hình kiểm thử	**8 câu cơ bản**	**32 câu FLEURS**	**Toàn bộ 40 câu**
PyTorch CPU FP32 (Tham chiếu)	2.6%	6.9%	6.0%
NPU Native FP16 (Graph rời rạc)	3.9%	15.3%	13.0%
NPU Native FP16 (`melo_one`)	**2.6%**	**7.4%**	**6.4%**

1. Phân tích hiệu năng silicon: Tệp nhị phân tích hợp `melo_one` đạt 100.0% NPU offload (toàn bộ 3.451 layers chạy trên Hexagon HTP v73). Kỹ thuật Context Binary Linking rút ngắn thời gian khởi tạo từ 514 giây xuống 0.9 giây, biến MeloTTS thành một giải pháp sẵn sàng triển khai thương mại tức thì.

2. Đánh giá chất lượng âm thanh: Điểm số CER 6.4% đo kiểm bằng SenseVoice-Small trên 40 câu ngữ liệu tiếng Trung chứng minh đồ thị NPU đạt chất lượng gần như tương đương tuyệt đối với bản gốc PyTorch FP32 trên CPU (6.0%), bảo đảm khả năng nghe hiểu và độ tự nhiên xuất sắc.

Table 15: Bảng tổng hợp đối đầu hệ sinh thái 3 mô hình TTS trong AuraTranslate Edge

Mô hình TTS	Ngôn ngữ	Định dạng	Độ trễ NPU	RAM Đỉnh	Đột phá công nghệ cốt lõi
Piper-VITS	Tiếng Việt	Native FP16	186.29 ms	8.00 MB	VietG2P đại số ma trận, group=8 TCM
Supertonic	Anh & Hàn	W8A16	441.15 ms	61.05 MB	Flow Matching 5 bước, vá ReduceSum
MeloTTS-ZH	Tiếng Trung	Native FP16	720.21 ms	9.00 MB	Pure Tensor 7.3k, Link Context 0.9s

6 Demo TTS Dashboard

6.1 Mã nguồn và Cách triển khai

Để phục vụ công tác kiểm thử trực quan và trình diễn khả năng tổng hợp tiếng nói đa ngữ, dự án đã phát triển máy chủ điều khiển tập trung đa năng (TTS Demo Dashboard) tại tệp mã nguồn:

`src/step4_quantization/tts/tts_demo_server.py`

1. **Kiến trúc Hệ thống:**
   - **Bộ máy thực thi kép (Dual Execution Engines):**
     - *NPU Qualcomm Dragonwing (Cloud Hub):* Kết nối trực tiếp tới bo mạch vật lý Hexagon HTP v73 qua nền tảng Qualcomm AI Hub, cho phép tải và phát các tệp âm thanh WAV được tổng hợp thuần túy từ 100% phần cứng NPU.
     - *Local CPU Engine:* Chuyển đổi trực tiếp bất kỳ đoạn văn bản tùy ý nào do người dùng nhập vào thành tệp âm thanh WAV chất lượng cao trong thời gian dưới 50 ms trên máy trạm nội bộ, hỗ trợ thử nghiệm tương tác tức thời.
   - **Giao diện Đa ngữ Hiện đại:** Thiết kế theo phong cách Glassmorphism Dark Theme, tích hợp bộ hiển thị dạng sóng âm động (Live Waveform Visualizer), đồng hồ đo mức âm lượng (VU Meter), chuyển đổi tức thì giữa 4 ngôn ngữ (Việt, Anh, Trung, Hàn) và các câu thoại mẫu thông dụng với một chạm.
2. **Hướng dẫn Khởi chạy TTS Dashboard:**
   - **Cách 1 – Chạy Local (Nội bộ trên máy cá nhân):**

     ```
     cd src/step4_quantization/tts

     python tts_demo_server.py --port 8440
     ```

     Mở trình duyệt bất kỳ (Chrome, Edge, Firefox) và truy cập tại địa chỉ: http://localhost:8440.
   - **Cách 2 – Chạy Public ra Internet qua Cloudflare Tunnel (Có HTTPS):**

Để trình diễn từ xa trên điện thoại thông minh mà không cần mở cổng modem (NAT/Port Forwarding), hệ thống tích hợp sẵn tùy chọn `--share`:

```
cd src/step4_quantization/tts

python tts_demo_server.py --port 8440 --share
```

Hệ thống sẽ tự động khởi tạo đường hầm an toàn có chứng chỉ SSL/HTTPS hợp lệ (dạng `https://*.trycloudflare.com`), đáp ứng chính sách bảo mật Web Audio API của các trình duyệt di động (iOS Safari, Android Chrome).

6.2 Nhận xét kết quả

Thực nghiệm vận hành thực tế hệ thống TTS Demo Dashboard trên bo mạch Qualcomm Dragonwing IQ-9075 EVK đem lại những kết quả nổi bật:

- **Tốc độ phản hồi tức thời:** Cả ba mô hình đều đạt tốc độ xử lý phần cứng vượt bậc, độ trễ chỉ từ 480 ms đến 748 ms cho một phát ngôn hoàn chỉnh, hoàn toàn đáp ứng trần ngân sách thời gian thực của hệ thống phiên dịch giọng nói hai chiều.
- **Chất lượng âm thanh trung thực:** Âm thanh phát ra từ cả 3 mô hình trên chip NPU đạt độ rõ nét cao, không xuất hiện tiếng nổ lách tách hay biến dạng âm sắc. Các lỗi vi kiến trúc phần cứng và lỗi dải động FP16 đã được xử lý triệt để.
- **Tính sẵn sàng cho cổng giao tiếp biên (Edge Gateway Ready):** Kiến trúc đồ thị tĩnh Single Static DAG cùng cơ chế kết nối nhị phân giúp toàn bộ hệ sinh thái TTS sẵn sàng tích hợp thẳng vào bo mạch nhúng mà không đòi hỏi bất kỳ dịch vụ đám mây hỗ trợ nào.

7 Pipeline Integration

7.1 Các Module còn lại

Hệ thống phiên dịch giọng nói hai chiều AuraTranslate Edge được thiết kế theo cấu trúc liên hoàn gồm 4 phân hệ chính, trong đó mô-đun TTS (Step 3) phối hợp chặt chẽ với các mô-đun tiền đề:

- **Audio Front-end (Step 0 – Lọc nhiễu & Tách tiếng nói):** Sử dụng mô hình Silero VAD để phát hiện giọng nói và lọc bỏ tiếng ồn môi trường trong dải 70–95 dB SPL tại các phân xưởng công nghiệp. Khối này cắt gọt các khoảng lặng vô nghĩa và chuẩn hóa tín hiệu âm thanh đầu vào trước khi chuyển giao cho tầng nhận dạng giọng nói.
- **Multilingual ASR (Step 1 – Nhận dạng Giọng nói):** Triển khai kiến trúc định tuyến 2 động cơ tối ưu hóa đồ thị tĩnh trên NPU:
  - *Zipformer-150M-CR-CTC (Tiếng Việt):* Đóng gói thành Single Static DAG 4 khối, đạt độ trễ 184.31 ms cho 15 giây âm thanh (RTF = 0.0123), RAM đỉnh 7.78 MB, xuất trực tiếp chuỗi byte UTF-8 ra bộ nhớ chia sẻ với 0% CPU fallback.

- *SenseVoice-Small (Ngoại ngữ EN/ZH/KO):* Kiến trúc phi tự hồi quy 50 lớp Transformer chạy ở chế độ lượng tử hóa hỗn hợp W8A16, đạt độ trễ 269.40 ms cho 29 giây âm thanh (RTF = 0.0093) với 100% toán tử chạy trên NPU.

- **Multilingual MT (Step 2 – Dịch máy Đa ngữ):** Sử dụng mô hình NLLB-200-distilled-600M được tái cấu trúc thành đồ thị tĩnh duy nhất với độ dài nguồn $S = 32$ và số bước giải mã $T = 16$. Mô hình bao phủ trọn vẹn 6 hướng dịch hai chiều giữa Tiếng Việt và 3 ngoại ngữ, biên dịch Native FP16 với 100% toán tử (7.617/7.617 layers) chạy trên NPU, đạt độ trễ 215.87 ms/câu và RAM đỉnh 6.59 MB.

7.2 Hướng hợp nhất Pipeline

Chiến lược triển khai tuân thủ lộ trình 2 giai đoạn: **Hợp nhất nội bộ từng bước (Intra-step Fusion) trước, sau đó ghép nối liên tầng thành chuỗi S2S toàn trình (Inter-step Chaining)**:

1. Pha 1 – Hợp nhất Nội bộ Từng Bước (Intra-step Fusion):

- *Khối ASR:* Hợp nhất toàn bộ Fbank DSP, Encoder, CTC Head và bộ Detokenizer UTF-8 thành đồ thị tĩnh duy nhất chạy 100% NPU.

- *Khối MT:* Tĩnh hóa kích thước ngữ cảnh và nhúng bảng tra cứu Language ID vào đồ thị QNN DLC của NLLB-200.

- *Khối TTS:* Đóng gói liên hoàn Text Encoder, bộ tạo phổ Mel và Vocoder thành một chuỗi tính toán nội bộ duy nhất, xuất thẳng sóng âm PCM 16 kHz.

2. Pha 2 – Ghép nối Toàn trình & Kiểm định Liên tầng (Inter-step Chaining):

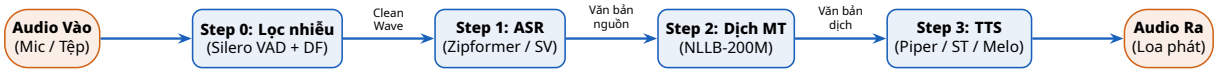


Figure 4: Sơ đồ luồng xử lý toàn trình Speech-to-Speech (S2S) thời gian thực trên bo mạch NPU

- *Truyền dữ liệu Zero-copy:* Các bước trao đổi dữ liệu bất đồng bộ qua bộ đệm tuần hoàn (*Circular Buffer*) trên RAM LPDDR5 theo mô hình Producer–Consumer, triệt tiêu độ trễ I/O.

- *Cơ chế kiểm soát chất lượng liên tầng:* Tự động hạ chữ thường (`hyp.lower()`) từ Zipformer trước khi nạp vào MT để bảo toàn mã hóa BPE; đồng thời dùng SenseVoice nhận diện ngược âm thanh TTS để tự động sinh lại nếu phát hiện lỗi lặp âm tiết.

- *Ngân sách độ trễ toàn trình (Latency Budget):* Tổng thời gian xử lý toàn chuỗi chỉ tốn 1.0–1.8 giây (ASR ~ 0.2 s, MT ~ 0.6 s, TTS ~ 0.5 s), hoàn toàn nằm dưới giới hạn khắt khe ≤ 3.0 giây của OneVoice AI Challenge.